

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	K.YOGANAND	Reg. No.:	192525173
Date:	07/10/2026	Duration:	60 minutes

Code of Conduct

I D Mahaboob Basha(192525369)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: D Basha_____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

Table of Contents

1. Understanding the Problem Statement

- 1.1 Problem Explained in Own Words
- 1.2 Objectives of the Proposed System
- 1.3 Scope of the Problem

2. Key Stakeholders and Challenges

- 2.1 Stakeholders
- 2.2 Roles and Responsibilities
- 2.3 Major Challenges and Issues

3. Analysis of the Current Approach

- 3.1 Existing System / Current Process
- 3.2 Strengths and Limitations
- 3.3 Problems and Gaps to be Addressed

4. Proposed Solution Using Software Engineering Principles

- 4.1 Overview and Architecture
- 4.2 Major Functional Modules
- 4.3 Functional and Non-Functional Requirements
- 4.4 Software Engineering Principles Incorporated

5. Justification of the Chosen Methodology

- 5.1 Selected Framework
- 5.2 Why It Is Suitable
- 5.3 Support for Development and Maintenance

6. Expected Outcomes and Limitations

- 6.1 Expected Benefits
- 6.2 How Challenges Are Addressed
- 6.3 Risks, Implementation Challenges and Limitations
- 6.4 Future Enhancements

7. Conclusion

8. References

1. Understanding the Problem Statement

1.1 Problem Explained in Own Words

AI-enabled applications cannot be evaluated only by checking whether their functions work correctly. An AI system may produce biased outcomes, expose private information, make decisions that are difficult to explain, consume excessive computing resources, or become less reliable as data and operating conditions change. The proposed problem is therefore to create a software platform that continuously audits AI-enabled software throughout its lifecycle. Instead of treating responsible AI as a one-time review, the platform connects requirements, design, development, testing, deployment, monitoring and maintenance with evidence, audit trails, reports and remediation actions.

The platform must provide visibility into privacy, fairness, security, explainability, performance, resource consumption and sustainability. It should also connect responsible-AI assessments with version control, CI/CD, testing, monitoring and project documentation.

1.2 Objectives of the Proposed System

- Track responsible-AI requirements throughout the software lifecycle.
- Evaluate privacy, fairness, security and explainability risks.
- Monitor model and application performance after deployment.
- Assess resource consumption and sustainability indicators.
- Integrate audits with testing and CI/CD workflows.
- Maintain evidence, audit trails and remediation actions.
- Support continuous maintenance and responsible evolution.

1.3 Scope of the Problem

In Scope	Out of Scope / Future Scope
Responsible-AI requirement tracking	Full replacement of existing enterprise governance systems
Privacy, fairness, security and explainability assessment	Legal certification or regulatory approval
Model and application performance monitoring	Training large foundation models from scratch
Resource consumption and sustainability metrics	Physical data-centre hardware management
Evidence collection and audit trails	Guaranteed elimination of all AI risks
Version-control and CI/CD integration	Automatic approval of production deployment without human governance
Testing and monitoring integration	Universal support for every development platform
Audit reporting and remediation tracking	Final organizational policy decisions

2. Key Stakeholders and Challenges

2.1 Stakeholders

- AI/ML developers and software developers
- Software testers and quality-assurance teams
- DevOps and platform engineers
- Project managers and product owners
- Responsible-AI, risk and governance teams
- Security and privacy teams
- Compliance officers and auditors
- Business owners and organizational management
- AI system users and affected stakeholders

2.2 Roles and Responsibilities

Stakeholder	Role and Responsibilities
AI/ML Developers	Build and update models, document datasets and model versions, address fairness and performance findings.
Software Developers	Implement application services, maintain code quality, security and lifecycle integration.
Testers / QA	Run functional, performance, security and responsible-AI tests and maintain test evidence.
DevOps / Platform Engineers	Integrate CI/CD, version control, deployment, monitoring, logging and infrastructure metrics.
Project Manager / Product Owner	Prioritize requirements, manage risks, coordinate stakeholders and approve remediation priorities.
Responsible-AI / Governance Team	Define responsible-AI criteria, review evidence, assess ethical risks and track governance actions.
Security / Privacy Team	Review data handling, access control, security risks, retention and privacy safeguards.
Auditor / Compliance Officer	Review audit trails, evidence, reports and compliance-related controls.
Management / Business Owner	Review overall risk posture, resource use, sustainability indicators and remediation progress.

2.3 Major Challenges and Issues

- Responsible-AI risks can appear at different lifecycle stages and may be missed by conventional functional testing.
- Bias and fairness issues may vary across datasets, models, user groups and versions.
- Privacy risks can arise from data collection, storage, access, retention and documentation.
- Opaque AI decisions make it difficult for developers, users and auditors to understand why an outcome occurred.
- Security weaknesses can occur in models, APIs, dependencies, infrastructure and deployment pipelines.
- Model and application performance may degrade after deployment because of changing data and operating conditions.
- Resource consumption may increase with model size, inference volume and infrastructure requirements.
- Sustainability indicators are difficult to track without consistent evidence and measurable metrics.
- Audit evidence may become fragmented across source control, CI/CD, test systems, monitoring tools and project documents.
- Remediation actions need ownership, priority, status and traceability across software versions.

3. Analysis of the Current Approach

3.1 Existing System / Current Process

Organizations may currently assess AI systems through separate activities such as software testing, security reviews, privacy reviews, model evaluation, project documentation, CI/CD checks and post-deployment monitoring. These activities may be distributed across different teams and tools rather than being managed as one lifecycle audit process.

A typical current process can be represented as: requirements and design → development → testing → deployment → monitoring → maintenance. Responsible-AI evidence may be collected at selected stages, but without a unified platform it can be difficult to maintain continuous traceability from a requirement or risk to its evidence, test result, software version and remediation action.

3.2 Strengths and Limitations

Current Approach	Strengths	Limitations
Manual / document-based audits	Flexible and can capture detailed review comments	Time-consuming, difficult to keep continuously updated, weak traceability
Separate testing and quality checks	Existing teams and tools can identify technical defects	Responsible-AI checks may be fragmented from lifecycle evidence
Version control and CI/CD	Provides version history and automated software delivery	Responsible-AI criteria may not be consistently linked to builds and releases
Post-deployment monitoring	Can detect performance and operational issues	May not cover fairness, explainability, sustainability or governance evidence
Periodic governance reviews	Provides management and compliance visibility	Issues can be detected late rather than continuously

3.3 Problems and Gaps to be Addressed

- No single lifecycle view of responsible-AI requirements, evidence and risks.
- Weak linkage between responsible-AI requirements and software versions, commits, tests and deployments.
- Fairness, privacy, security and explainability assessments may be performed inconsistently.
- Resource consumption and sustainability indicators may not be measured continuously.
- Audit evidence can be distributed across documents, repositories, CI/CD tools and monitoring systems.
- Remediation actions may not have clear ownership, priority, status and closure evidence.
- Model degradation after deployment may not be connected to responsible-AI review workflows.
- Audit reports may require manual consolidation from multiple sources.

4. Proposed Solution Using Software Engineering Principles

4.1 Overview and Architecture

The proposed platform is a centralized, API-enabled Responsible AI Software Lifecycle and Sustainability Audit Platform. It connects lifecycle requirements with evidence from source control, development, testing, CI/CD, deployment, monitoring and project documentation. The platform evaluates privacy, fairness, security, explainability, performance, resource consumption and sustainability, then produces audit reports and tracks remediation.

Proposed logical flow:

- Project / AI System → Responsible-AI Requirements Register
- Requirements → Design and Development Evidence
- Source Control + CI/CD + Testing → Automated Evidence Collection
- Deployment + Monitoring → Performance, Risk and Resource Metrics
- Audit Engine → Privacy / Fairness / Security / Explainability / Sustainability Assessment
- Findings → Risk Rating → Remediation Assignment → Verification
- Evidence + Findings + Remediation → Audit Report and Lifecycle Traceability

4.2 Major Functional Modules

Module	Main Function	Key Outputs
1. Responsible-AI Requirements Module	Capture and track responsible-AI requirements throughout the lifecycle.	Requirement register, ownership, status and traceability.
2. Evidence & Audit Module	Collect and organize evidence for privacy, fairness, security, explainability and governance.	Evidence records, audit trail and assessment results.
3. Testing & CI/CD Integration Module	Connect responsible-AI checks with automated testing, builds and deployment workflows.	Test results, pipeline evidence and release status.
4. Monitoring & Sustainability Module	Monitor model/application performance, resource consumption and sustainability indicators after deployment.	Metrics, alerts, trends and sustainability records.
5. Risk & Remediation Module	Record findings, assign risk levels and track corrective actions to closure.	Risk register, remediation tracker and verification evidence.
6. Reporting & Dashboard Module	Present lifecycle status, audit findings, evidence coverage and sustainability results.	Audit reports, dashboards and management summaries.

4.3 Functional and Non-Functional Requirements

Functional Requirements

ID	Requirement
FR1	Create and track responsible-AI requirements across lifecycle stages.
FR2	Record privacy, fairness, security and explainability assessments.
FR3	Collect evidence from project documentation, version control, testing and CI/CD.
FR4	Record model and application performance monitoring results.
FR5	Capture resource consumption and sustainability indicators.
FR6	Maintain audit trails linking requirements, evidence, findings and software versions.
FR7	Create, assign, prioritize and track remediation actions.
FR8	Generate audit reports and lifecycle dashboards.
FR9	Record monitoring findings and model/application degradation events.
FR10	Support continuous updates as software and AI models evolve.

Non-Functional Requirements

Attribute	Requirement
Security	Role-based access, secure integration, audit protection and controlled evidence access.
Performance	Audit and dashboard operations should provide timely results for normal project workloads.
Scalability	The platform should support multiple AI projects, versions, evidence items and lifecycle events.
Reliability	Evidence and audit records should remain available and consistent during normal operation.
Usability	Dashboards and workflows should be understandable to developers, auditors and management.
Maintainability	Modular services, documented APIs, version control and automated tests should support change.
Traceability	Requirements, evidence, findings, versions, tests and remediation actions should be linkable.
Sustainability	The platform should support measurable resource-consumption and sustainability indicators.

4.4 Software Engineering Principles Incorporated

Principle	How It Is Incorporated
Modularity	Separate modules for requirements, evidence, audits, testing/CI-CD, monitoring, remediation and reporting.
Scalability	API-based and service-oriented design that can support multiple projects and lifecycle events.
Maintainability	Version-controlled code, documented interfaces, automated testing and configurable assessment rules.
Reliability	Validation, logging, error handling, backups and monitoring for critical platform services.
Usability	Role-based dashboards, clear risk status, evidence views and simple remediation workflows.
Security	Authentication, authorization, secure data handling, logging and least-privilege access.
Testability	Automated unit, integration and responsible-AI checks connected to CI/CD.
Traceability	Links between requirements, commits, builds, tests, deployments, evidence and remediation.
Ethics and Accountability	Explicit privacy, fairness, explainability and governance assessment with audit evidence.

5. Justification of the Chosen Methodology

5.1 Selected Framework

The selected approach is Agile (Scrum) combined with DevOps practices, including continuous integration and continuous delivery. Work can be divided into short iterations so that lifecycle auditing, evidence integration, monitoring and sustainability features can be developed and evaluated incrementally.

Sprint	Goal
Sprint 0	Requirements, architecture, repository and CI/CD setup
Sprint 1	Responsible-AI requirements and evidence management
Sprint 2	Audit engine and privacy/fairness/security/explainability assessments
Sprint 3	Testing and CI/CD integration
Sprint 4	Monitoring, resource consumption and sustainability module
Sprint 5	Risk, remediation, reporting, security hardening and documentation

5.2 Why It Is Suitable

- Evolving AI risks require iterative assessment and the ability to change requirements.
- Responsible-AI evaluation benefits from early prototypes and repeated testing rather than late-stage assessment.
- The platform has multiple independent modules that can be delivered incrementally.
- CI/CD integration requires continuous automated testing and evidence generation.
- Monitoring and sustainability indicators need continuous improvement after deployment.
- Stakeholders such as developers, auditors, security teams and project owners can provide feedback during sprint reviews.

5.3 Support for Development and Maintenance

- Development: user stories and responsible-AI requirements can be prioritized in a backlog.
- Continuous integration: commits can trigger automated software tests and responsible-AI checks.
- Continuous delivery: validated versions can move through controlled environments with evidence retained.
- Maintenance: monitoring findings can create remediation tasks and improvement backlog items.
- Traceability: requirements can be linked to versions, tests, evidence, findings and remediation actions.
- Retrospectives can be used to improve audit workflows, assessment rules and sustainability measurement.

6. Expected Outcomes and Limitations

6.1 Expected Benefits

- Improve responsible development of AI-enabled software.
- Provide continuous visibility of AI risks across the lifecycle.
- Strengthen software quality and governance.
- Support measurable sustainability assessment.
- Improve traceability of ethical and technical decisions.
- Enable safer and more sustainable evolution of AI applications.
- Reduce fragmentation by connecting lifecycle evidence and audit activities.
- Improve remediation tracking and accountability.

6.2 How Challenges Are Addressed

Challenge Identified	How the Proposed Solution Addresses It
Fragmented responsible-AI activities	Central lifecycle requirements, evidence, findings and reports.
Bias and fairness risks	Fairness assessment records, evidence and remediation tracking.
Privacy concerns	Privacy requirements, evidence, controlled access and audit trails.
Opaque decisions	Explainability assessments and documented evidence for model/application decisions.
Security risks	Security checks integrated with development and CI/CD evidence.
Performance degradation	Post-deployment monitoring and lifecycle alerts.
Excessive resource consumption	Resource metrics and sustainability indicators.
Weak traceability	Links among requirements, versions, tests, deployments, findings and remediation.
Manual audit reporting	Centralized evidence and automated/structured audit reporting.

6.3 Risks, Implementation Challenges and Limitations

Risk / Limitation	Possible Mitigation
Incomplete or inconsistent evidence	Define evidence templates, ownership and lifecycle checkpoints.
Changing AI models and requirements	Use versioned assessment criteria and configurable audit rules.
Integration complexity	Start with commonly used version control, CI/CD, testing and monitoring interfaces.
Measurement difficulty for sustainability	Define a consistent set of measurable resource and sustainability indicators.

False confidence from audit scores	Use audit results as evidence-based decision support rather than proof that all risks are eliminated.
Large evidence volume	Use structured storage, metadata, retention rules and searchable audit records.
Limited prototype validation	Test with representative project scenarios and clearly document prototype limitations.
Security of audit evidence	Apply access control, secure storage, logging and appropriate retention policies.

6.4 Future Enhancements

- Add broader integrations with enterprise version-control, CI/CD, testing and monitoring platforms.
- Introduce configurable responsible-AI assessment frameworks for different application domains.
- Add advanced model-drift and fairness-drift detection.
- Develop richer sustainability dashboards and trend analysis.
- Add automated evidence validation and duplicate-evidence detection.
- Introduce stronger policy-as-code controls for CI/CD governance.
- Support organization-wide portfolio views across multiple AI applications.
- Add advanced analytics for recurring risks and remediation effectiveness.

7. Conclusion

AI-enabled software requires lifecycle governance beyond conventional functional testing. The proposed Responsible AI Software Lifecycle and Sustainability Audit Platform provides a structured approach for tracking responsible-AI requirements, collecting evidence, assessing privacy, fairness, security and explainability, monitoring performance, measuring resource consumption and sustainability, and managing remediation. By integrating version control, CI/CD, testing, monitoring and project documentation, the platform improves traceability from requirements to evidence and corrective actions. An Agile and DevOps approach supports incremental development, continuous assessment and responsible evolution of the platform.

8. References

- Sommerville, I., Software Engineering, 10th ed., Pearson, 2016.
- Pressman, R. S., Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill, 2014.
- Schwaber, K. and Sutherland, J., The Scrum Guide, 2020.
- NIST, Artificial Intelligence Risk Management Framework (AI RMF 1.0), 2023.
- ISO/IEC 42001, Artificial Intelligence Management System standard.
- OWASP, Application Security Verification Standard and software security guidance.

RUBRICS FOR EVALUATION:

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses	Limited analysis with partial identification of	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25